



河南大學
HENAN UNIVERSITY



Web编程基础

(2020-2021-2)

任课教师: 丁爽



内容回顾1

- 事件是一种“**触发-响应**”的机制；事件三要素：事件源、事件类型、事件处理程序
- 操作元素：
 - 操作元素内容：`element.innerHTML`、`element.innerText`、`element.textContent`
 - 操作元素属性：**src**、**title**...
 - 操作元素样式：**style**属性；**className**属性



内容回顾2

- 排他思想：所有元素全部清除，设置当前元素
- 属性操作：
 - 获取属性值：`element.属性`, `getAttribute('属性')`
 - 设置属性值：`element.属性= '值'`, `element.setAttribute('属性', '值')`
 - 移除属性值：`element.removeAttribute('属性')`
- 自定义属性
 - 设置属性值：`data-属性名`, `setAttribute('属性', 值)`, `元素对象.dataset.属性名= '值'`
 - 获取属性值：`getAttribute()`, `element.dataset.属性` 或者 `element.dataset['属性']`



7.4 节点基础

1

什么是节点

HTML文档可以看作是一个节点树，网页中的所有内容都是[节点](#)，一般来说，节点至少拥有`nodeType`（节点类型）、`nodeName`（节点名称）和`nodeValue`（节点值）这3个基本属性。下面列举常见的节点类型：

- ❑ 元素节点，`nodeType`为1
- ❑ 属性节点，`nodeType`为2
- ❑ 文本节点，`nodeType`为3，文本节点包含文字、空格、换行等



7.4 节点基础

2 节点层级

一个HTML文件可以看作是所有元素组成的一个节点树，各元素节点之间有级别的划分。

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>测试</title>
  </head>
  <body>
    <a href="#">链接</a>
    <p>段落...</p>
  </body>
</html>
```

DOM根据HTML中各节点的不同作用，可将其分别划分为标签节点（元素节点）、文本节点和属性节点。节点之间的层级关系如下：

- ❑ 根节点：<html>标签是整个文档的根节点，有且仅有一个
- ❑ 父节点：指的是某一个节点的上级节点
- ❑ 子节点：指的是某一个节点的下级节点
- ❑ 兄弟节点：两个节点同属于一个父节点



7.4 节点基础

2

节点层级

① 获取父级节点

例7.4.2-2

在JavaScript中，可以使用parentNode属性来获得离当前元素的最近的一个父节点，如果找不到父节点就返回为 null，语法格式为：obj.parentNode，obj是一个DOM对象。



7.4 节点基础

2

节点层级

案例演示: 如何获取当前元素的父节点。

示例代码

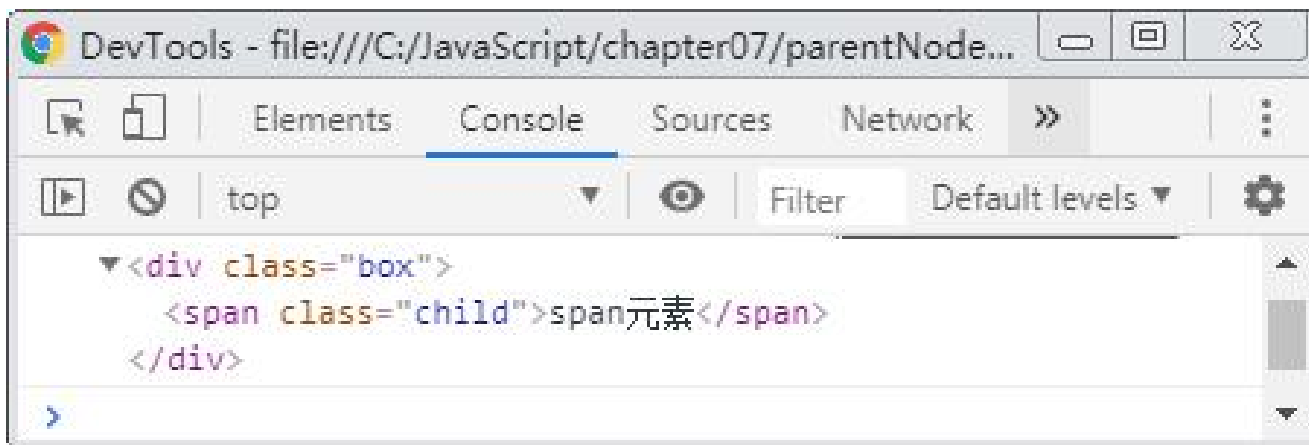
```
<body>
  <div class="demo">
    <div class="box"> <span class="child">span元素</span> </div>
  </div>
  <script>
    var child = document.querySelector('.child'); // 获取类名为child的span元素
    console.log(child.parentNode); // 输出离child元素最近的父级节点(box)
  </script>
</body>
```

7.4 节点基础

2

节点层级

浏览器预览效果图如下所示：



父级节点



7.4 节点基础

2

节点层级

② 获取子级节点

在JavaScript中，可以使用childNodes属性或者children属性两种方式来获得当前元素的所有子节点的集合，接下来我们就分别介绍这两种方式的用法。

方式1: `childNodes`属性获得的是当前元素的所有子节点的集合，该集合为即时更新的集合。下面通过案例的形式演示如何获取当前元素子节点。



7.4 节点基础

2

节点层级

例7.4.2-3

案例演示: 如何使用childNodes属性获取当前元素的子节点。

示例代码

```
<ul><li>我是li</li><li>我是li</li><li>我是li</li><li>我是li</li></ul>
<script>
  var ul = document.querySelector('ul');
  var lis = ul.querySelectorAll('li');
  console.log(lis)
  console.log(ul.childNodes);
  console.log(ul.childNodes[0].nodeType);
  console.log(ul.childNodes[1].nodeType);
</script>
```



7.4 节点基础

2

节点层级

注意

`childNodes`属性返回的是`NodeList`对象的集合，返回值里面包含了元素节点、文本节点等其他类型的节点。

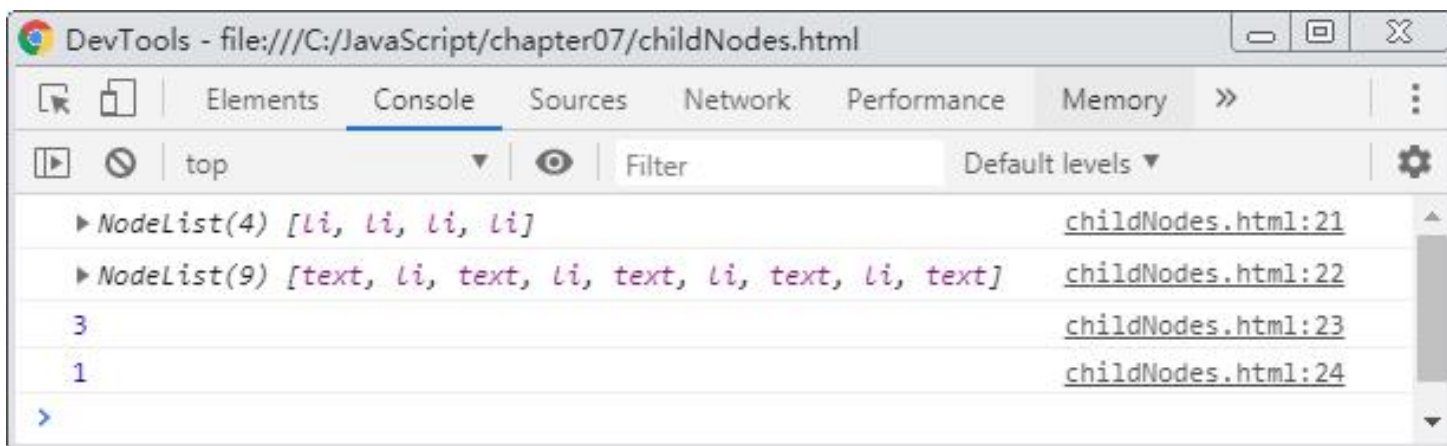


7.4 节点基础

2

节点层级

浏览器预览效果图如下所示：



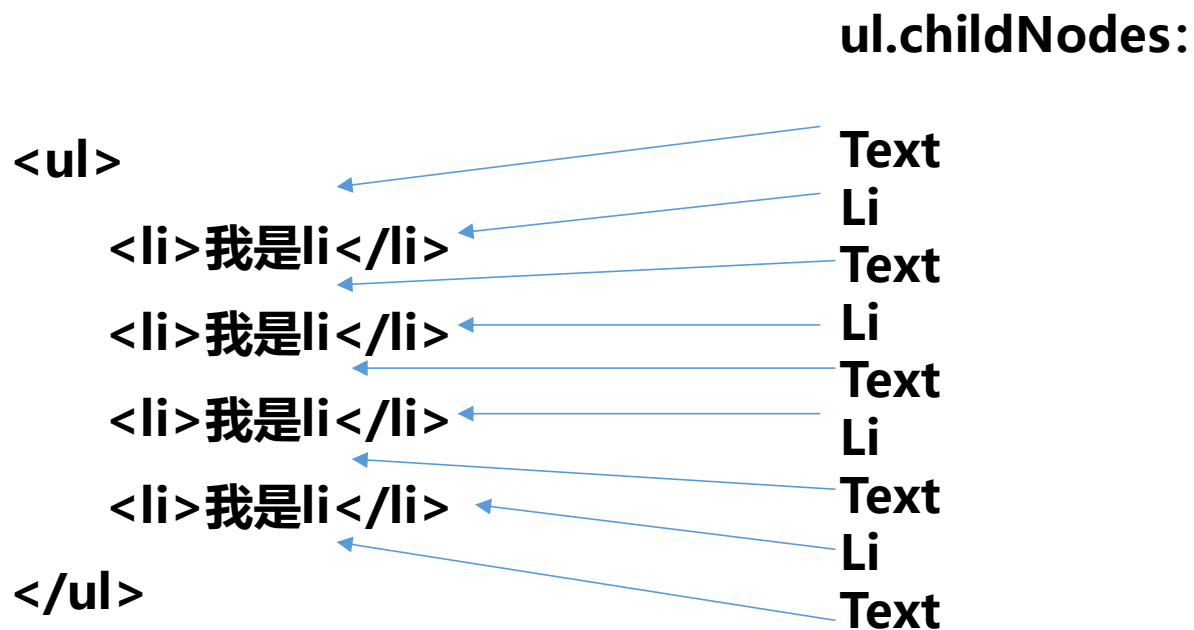
子级节点



7.4 节点基础

2

节点层级





7.4 节点基础

2

节点层级

方式2: `children`是一个可读的属性，返回所有子元素节点。`children`只返回子元素节点，其余节点不返回，目前各大浏览器都支持该属性，在实际开发中推荐使用`children`。

接下来，通过案例的形式演示如何获取当前元素子节点。



7.4 节点基础

2

节点层级

例7.4.2-4

案例演示：如何使用children属性获取当前元素的子节点。

HTML代码

```
<ol>
  <li>我是li</li>
  <li>我是li</li>
  <li>我是li</li>
  <li>我是li</li>
</ol>
```

JS代码

```
<script>
  var ul = document.querySelector('ol');
  var lis = ul.querySelectorAll('li');
  console.log(ul.children); // 结果为：HTMLCollection(4) [li, li, li, li]
</script>
```

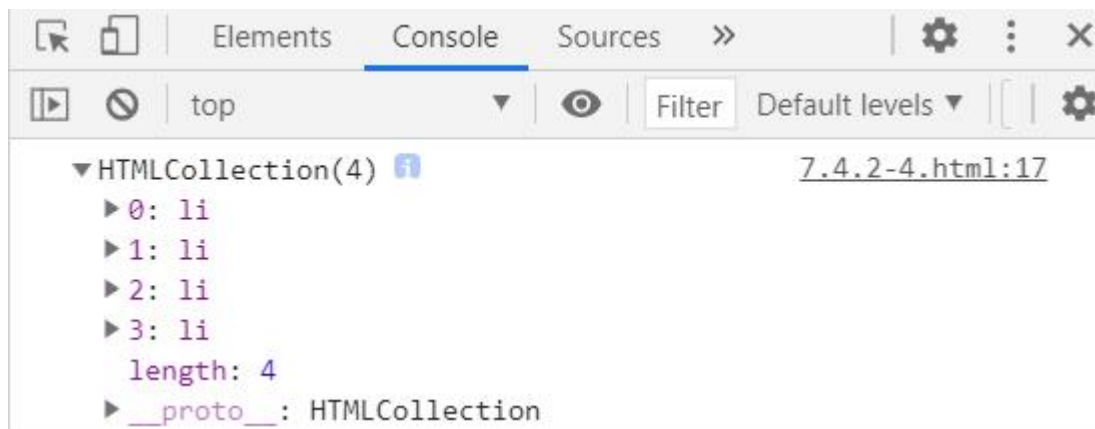


7.4 节点基础

2

节点层级

浏览器预览效果图如下所示：



子级节点



7.4 节点基础

2

节点层级

值得一提

`childNodes`属性与`children`属性虽然都可以获取某元素的子元素，但是两者之间有一定的区别。前者用于节点操作，返回值是`NodeList`对象集合，后者用于元素操作，返回的是`HTMLCollection`对象集合。



7.4 节点基础

2

节点层级

方式3: 使用firstChild属性和lastChild属性[获取子节点](#)，前者返回第一个子节点，后者返回的是最后一个子节点，如果找不到则返回null。需要注意的是它们的返回值包括文本节点和元素节点等。



7.4 节点基础

2

节点层级

方式4: 使用firstElementChild属性和lastElementChild属性获取[子元素节点](#)，前者返回第一个子元素节点，后者返回最后一个子元素节点，如果找不到则返回null。需要注意的是，**这两个属性有兼容性问题**，IE9以上才支持。



7.4 节点基础

2

节点层级

注意

实际开发中，`firstChild` 和 `lastChild` 包含其他节点，操作不方便，而 `firstElementChild` 和 `lastElementChild` 又有兼容性问题，为了解决兼容性问题，在实际开发中通常使用 “`obj.children[索引]`” 的方式来获取子元素节点。

示例代码

```
obj.children[0]
```

```
// 获取第一个子元素节点
```

```
obj.children[obj.children.length - 1]
```

```
// 获取最后一个子元素节点
```



7.4 节点基础

2

节点层级

③ 获取兄弟节点

在JavaScript中，可以使用[nextSibling](#)属性获得下一个兄弟节点，或者使用[previousSibling](#)属性获得上一个兄弟节点，它们的返回值包含元素节点或者文本节点等。如果找不到，就返回null。



7.4 节点基础

2

节点层级

可以使用`nextElementSibling`返回当前元素的下一个兄弟元素节点，`previousElementSibling`属性返回当前元素的上一个兄弟元素节点，如果找不到则返回`null`。需要注意的是，这两个属性有兼容性问题，IE9以上才支持。



7.4 节点基础

2

节点层级

实际开发中，`nextSibling`和`previousSibling`属性返回值都包含其他节点，操作不方便，而`nextElementSibling`和`previousElementSibling`又有兼容性问题。

为了解决兼容性问题，在实际开发中通常使用封装函数来处理兼容性。



7.4 节点基础

2

节点层级

示例代码

```
function getNextElementSibling(element) {  
  var el = element;  
  while (el = el.nextSibling) {  
    if (el.nodeType === 1) {  
      return el;  
    }  
  }  
  return null;  
}
```




7.4 节点基础

3

【案例】下拉菜单

例7.4.3

案例需求: 鼠标指针经过菜单时，显示当前下拉框中的内容同时隐藏其他下拉菜单内容

① 编写HTML结构代码

示例代码

```
<body>
  <ul class="nav">
    <li>
      <a href="#">微博</a>
      <ul></ul>
    </li>
    ... (此处省略3个li)
  </ul>
</body>
```

a为标签名，ul为下拉内容



7.4 节点基础

3

【案例】下拉菜单

② 鼠标经过，展示当前下拉列表内容

示例代码

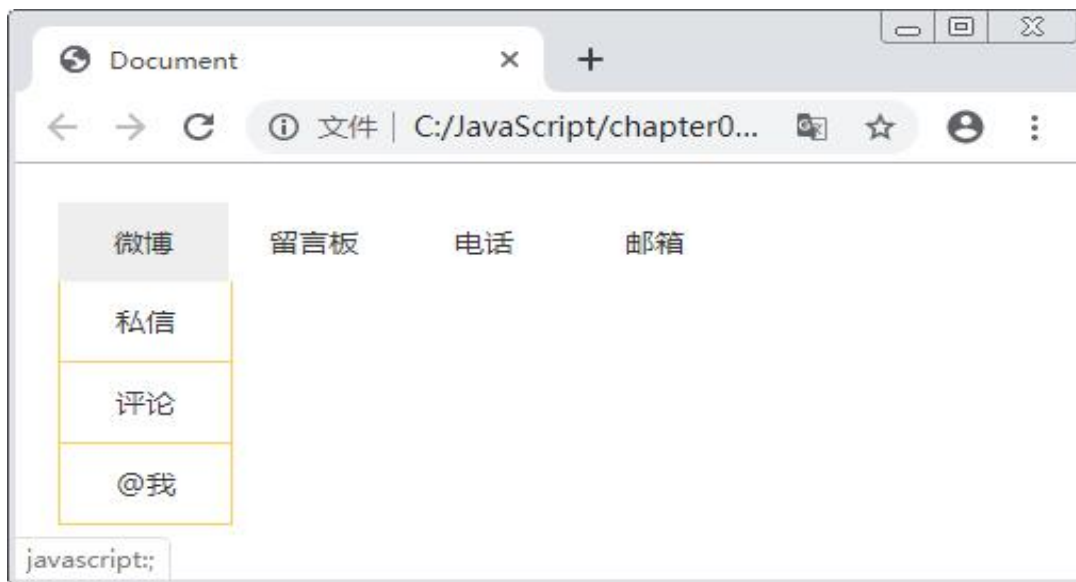
```
<script>
var nav = document.querySelector('.nav'); // 1. 获取元素
var lis = nav.children;
for (var i = 0; i < lis.length; i++)
lis[i].onmouseover = function () { // 2. 注册鼠标指针经过和鼠标指针离开事件
    this.children[1].style.display = 'block'; // ul为li的第2个子元素
};
lis[i].onmouseout = function () { this.children[1].style.display = 'none'; };
}
</script>
```

7.4 节点基础

3

【案例】下拉菜单

浏览器预览效果图如下所示：



下拉菜单效果



7.5 节点操作

1

创建节点

在DOM中，使用[document.createElement\('tagName'\)](#)方法创建由tagName指定的HTML元素，也称为动态创建元素节点。

动态创建元素节点的3种常见方式如下：

- ❑ document.write()创建元素，如果页面文档流加载完毕，再调用会导致页面重绘
- ❑ element.innerHTML将内容写入某个DOM节点，不会导致页面全部重绘
- ❑ document.createElement()创建多个元素效率稍微低一点，但是结构更加清晰



7.5 节点操作

2

添加和删除节点

DOM中，提供了[node.appendChild\(\)](#)和[node.insertBefore\(\)](#)方法用于添加节点，[node.removeChild\(child\)](#)用于删除节点。

下面讲解这3种方法的使用。

- ❑ `appendChild()`方法，将一个节点添加到指定父节点的子节点列表末尾
- ❑ `insertBefore(child, 指定元素)`方法，将一个节点添加到父节点的指定子节点前面
- ❑ `removeChild(child)`用于删除节点，该方法从DOM中删除一个子节点，返回删除的节点



7.5 节点操作

3

【案例】简易留言板

例7.5.3

案例分析：利用节点的创建、添加和删除相关知识完成一个简易的留言板功能。在页面中实现单击“发布”按钮动态创建一个li元素，添加到ul里面。案例实现步骤如下：



7.5 节点操作

3

【案例】简易留言板

① 编写HTML结构，完成页面布局

示例代码

```
<body>  
  <textarea name="" id=""></textarea> // 用户输入留言  
  <button>发布</button>  
  <ul></ul> // 用于展示留言模块  
</body>
```



7.5 节点操作

3

【案例】简易留言板

② 添加留言功能: 获取元素

示例代码

```
<script>  
  // 1. 获取元素  
  var btn = document.querySelector('button');  
  var text = document.querySelector('textarea');  
  var ul = document.querySelector('ul');  
</script>
```




7.5 节点操作

3

【案例】简易留言板

② 添加留言功能: 注册事件, 添加留言

示例代码

```
btn.onclick = function () {  
    if (text.value == '') {  
        alert('您没有输入内容');  
        return false;  
    } else { // (1) 创建元素  
        var li = document.createElement('li');  
        li.innerHTML = text.value;  
        ul.insertBefore(li, ul.children[0]); // (2) 添加元素  
        // ... (3) 此处编写删除留言功能  
    }  
};
```

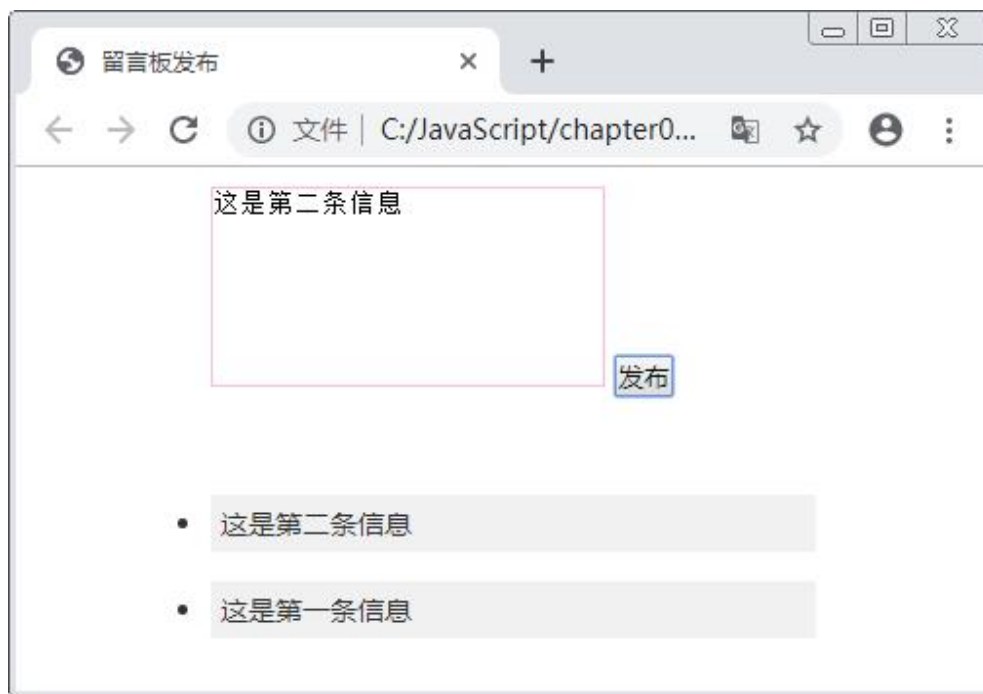


7.5 节点操作

3

【案例】简易留言板

浏览器预览效果图如下所示：



添加留言



7.5 节点操作

3

【案例】简易留言板

- ③ 删除留言功能: 首先在添加li时, 给li中增加一个a链接, 给所有的a链接注册单击事件, 找到a的父节点li, 进行删除即可。

修改第 (2) 步中代码, 找到如下所示代码

```
li.innerHTML = text.value;
```

修改为

```
li.innerHTML = text.value + '<a href="javascript:;">删除</a>';
```



7.5 节点操作

3

【案例】简易留言板

③ 删除留言功能: 注册事件, 删除留言

示例代码

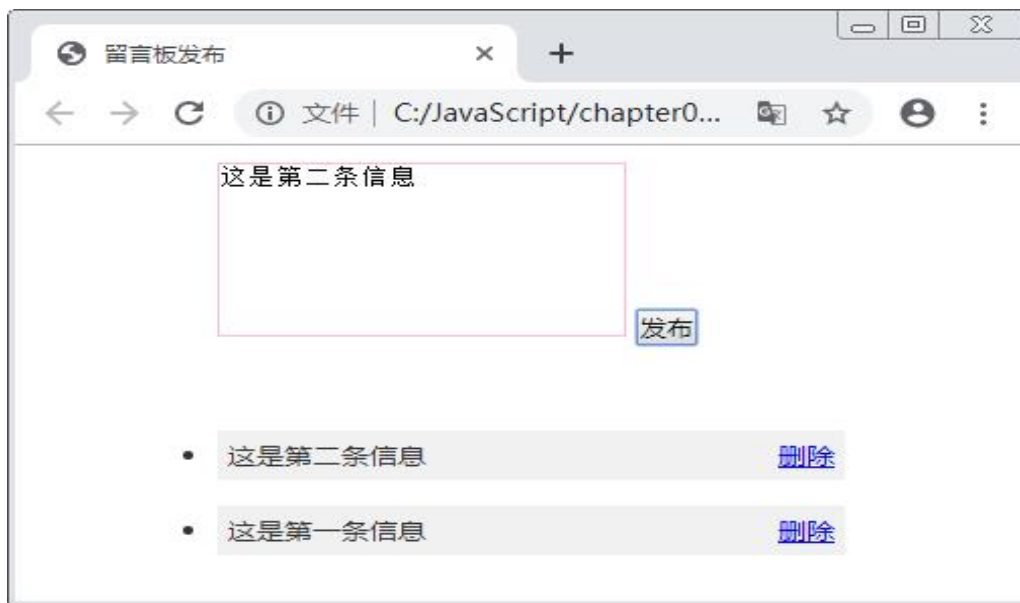
```
// 在(3)处编写以下代码
var as = document.querySelectorAll('a');
for (var i = 0; i < as.length; i++) {
  as[i].onclick = function () {
    ul.removeChild(this.parentNode);
  };
}
```

7.5 节点操作

3

【案例】简易留言板

浏览器预览效果图如下所示：



删除留言



7.5 节点操作

4

复制节点

在DOM中，提供了[node.cloneNode\(\)](#)方法，返回调用该方法的节点的一个副本，也称为克隆节点或者拷贝节点。语法为“**需要被复制的节点.cloneChild(true/false)**”。如果参数为空或**false**，则是**浅拷贝**，即**只复制节点本身，不复制里面的子节点**；如果括号参数为**true**，则是**深拷贝**，即会**复制节点本身及里面所有的子节点**。



7.5 节点操作

4

复制节点

例7.5.4

示例代码

```
<body>
  <ul id="myList"> <li>苹果</li> <li>橙子</li> <li>橘子</li> </ul>
  <ul id="op"> </ul>
  <button onclick="myFunction()">点我</button>
  <script>
    function myFunction() {
      var item = document.getElementById('myList').firstChild;
      var cloneItem = item.cloneNode(true);
      document.getElementById('op').appendChild(cloneItem);
    }
  </script>
</body>
```

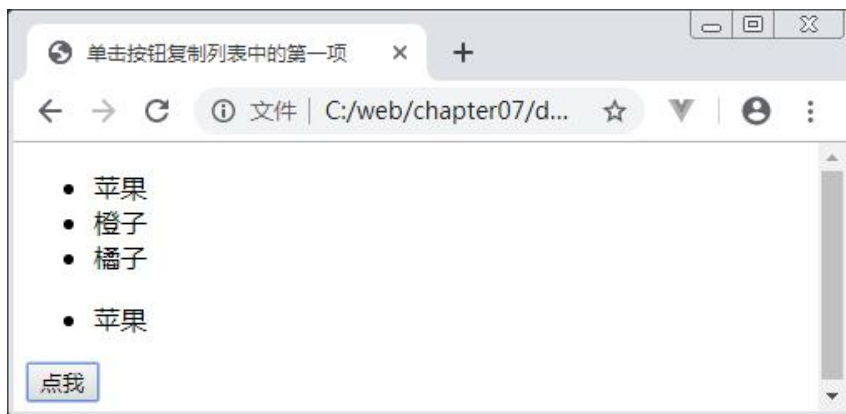


7.5 节点操作

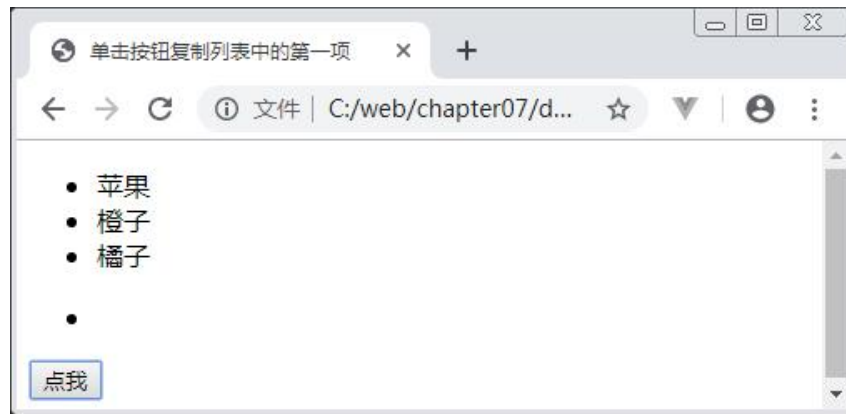
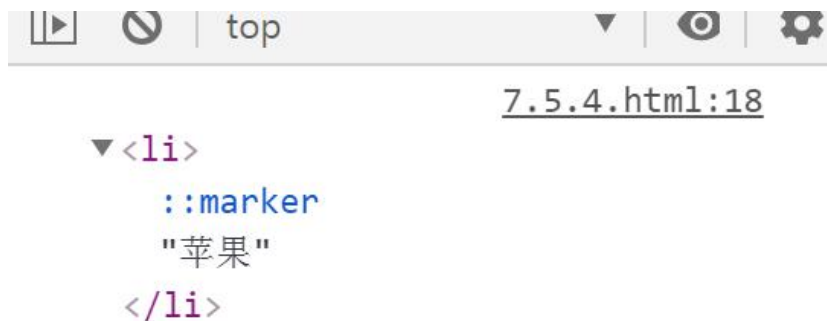
4

复制节点

当单击“点我”按钮的时候，浏览器预览效果图如下所示：



cloneNode(true)



cloneNode(false)





7.6 事件进阶

0

注册多种表示方式

<body>

<button id="myclose">关闭</button>

<script type="text/javascript">

var myclose=document.

getElementById("myclose")

myclose.onclick=function(){

window.close();

}

</script>

</body>

<body>

<button onclick="myclose()">关闭</button>

<script type="text/javascript">

function myclose(){

window.close();

}

</script>

</body>

2

<body>

<button onclick="window.close()">关闭</button>

</body>

3



7.6 事件进阶

1

注册事件

在JavaScript中，注册事件（绑定事件）有两种方式，即[传统方式注册事件](#)和[事件监听](#)方式注册事件。下面分别针对这两种方式进行讲解。



7.6 事件进阶

1

注册事件

① 传统方式

语法结构

```
元素对象.事件 = 事件的处理程序;  
// 示例  
oBtn.onclick = function () { }
```

注意: 该方式注册事件的特点在于注册事件的唯一性，即同一个元素同一个事件只能设置一个处理函数，最后注册的处理函数将会覆盖前面注册的处理函数。



7.6 事件进阶

1

注册事件

② 事件监听方式

由于不同浏览器采用的事件流实现方式不同，事件监听的实现存在兼容性问题。

通常根据浏览器的内核，可以把浏览器划分为两大类，一类是早期版本的IE浏览器（如IE 6~IE 8），一类是遵循W3C标准的浏览器（以下简称标准浏览器）。

注意：该方式注册事件的特点在于可以给同一个DOM对象的同一个事件添加多个事件处理程序。

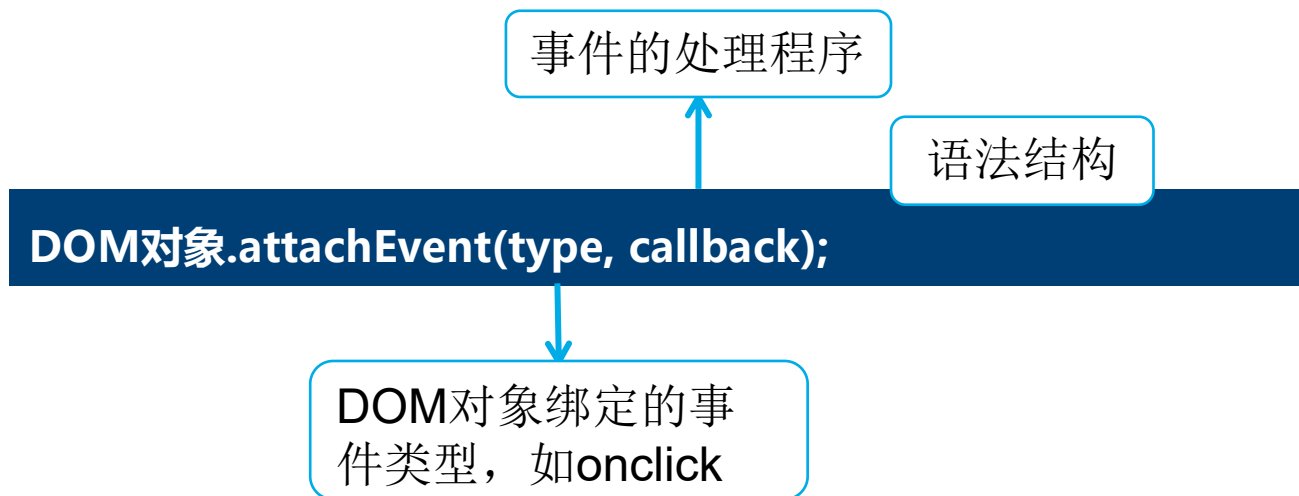


7.6 事件进阶

1

注册事件

早期IE内核的浏览器，事件监听方式如下：



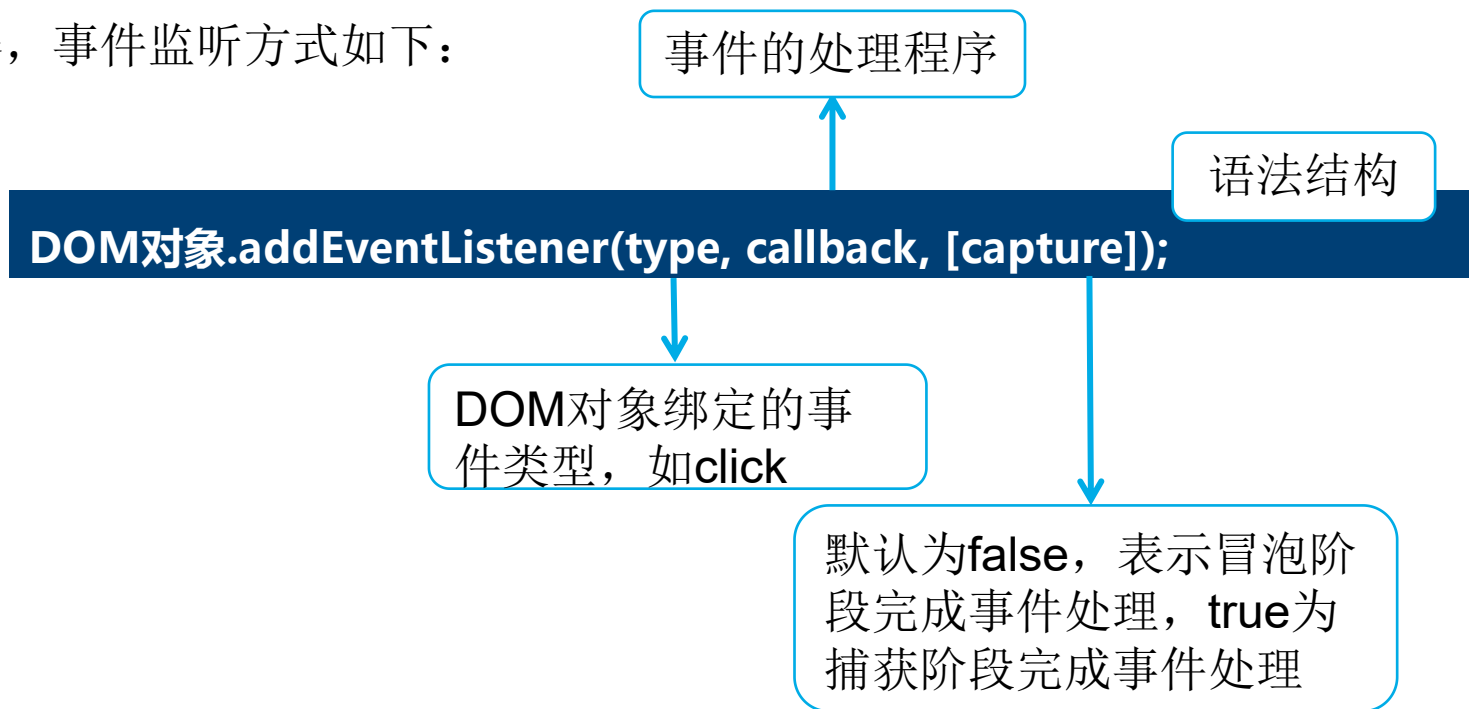


7.6 事件进阶

1

注册事件

标准浏览器，包括IE8版本以上的IE浏览器，以及新版的Firefox、Chrome等浏览器，事件监听方式如下：





7.6 事件进阶

1

注册事件

capture取值

false: 冒泡阶段完成事件处理

true: 捕获阶段完成事件处理

eg1-冒泡

```
<body>
  <div class="one">
    我是父亲
    <div class="two">
      我是儿子
    </div>
  </div>
</body>
<script type="text/javascript">
  var a = document.querySelector('.one')
    .addEventListener('click', function() {
      console.log('我是父亲')
    },true)
  var b = document.querySelector('.two')
    .addEventListener('click', function(e) {
      console.log('我是儿子');
      // e.stopPropagation();
    },true)
</script>
</body>
```



7.6 事件进阶

1

注册事件

例7.6.1-1

```
<div id="t">test</div>
<script>
  var obj = document.getElementById('t');
  // 添加第1个事件处理程序
  obj.attachEvent('onclick', function () {
    console.log('one');
  });
  // 添加第2个事件处理程序
  obj.attachEvent('onclick', function () {
    console.log('two');
  });
</script>
```

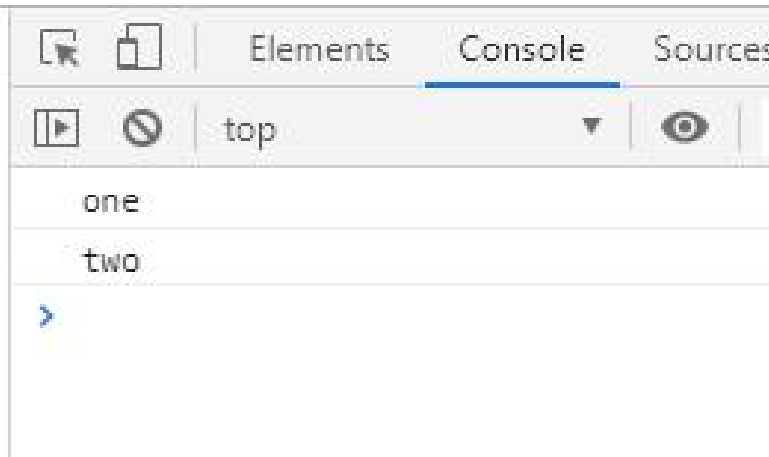
早期版本IE执行
按照添加顺序倒序执行

结果：
two
one

7.6 事件进阶



/chapter07/7.6.1-2.html



标准浏览器执行
按照添加顺序正序执行

例7.6.1-2

1 注册事件

```
<div id="t">test</div>
<script>
  var obj = document.getElementById('t');
  // 添加第1个事件处理程序
  obj.addEventListener('click', function () {
    console.log('one');
  });
  // 添加第2个事件处理程序
  obj.addEventListener('click', function () {
    console.log('two');
  });
</script>
```



7.6 事件进阶

2

删除事件

事件监听的移除也需要考虑兼容性问题，示例代码如下。

语法结构

```
DOM对象.onclick = null;           // 传统方式删除事件
DOM对象.detachEvent(type, callback); // 早期版本IE浏览器
DOM对象.removeEventListener(type, callback); // 标准浏览器
```

保证事件监听的处理程序是一个有名字的函数

```
obj.addEventListener('click', myfun);
function myfun(){
    console.log('three');
};
obj.removeEventListener('click', myfun);
```



7.6 事件进阶

3

DOM事件流

事件流是指当事件发生时，会在发生事件的元素节点与DOM树根节点之间按照特定的顺序进行传播，这个过程称之为**事件流**。

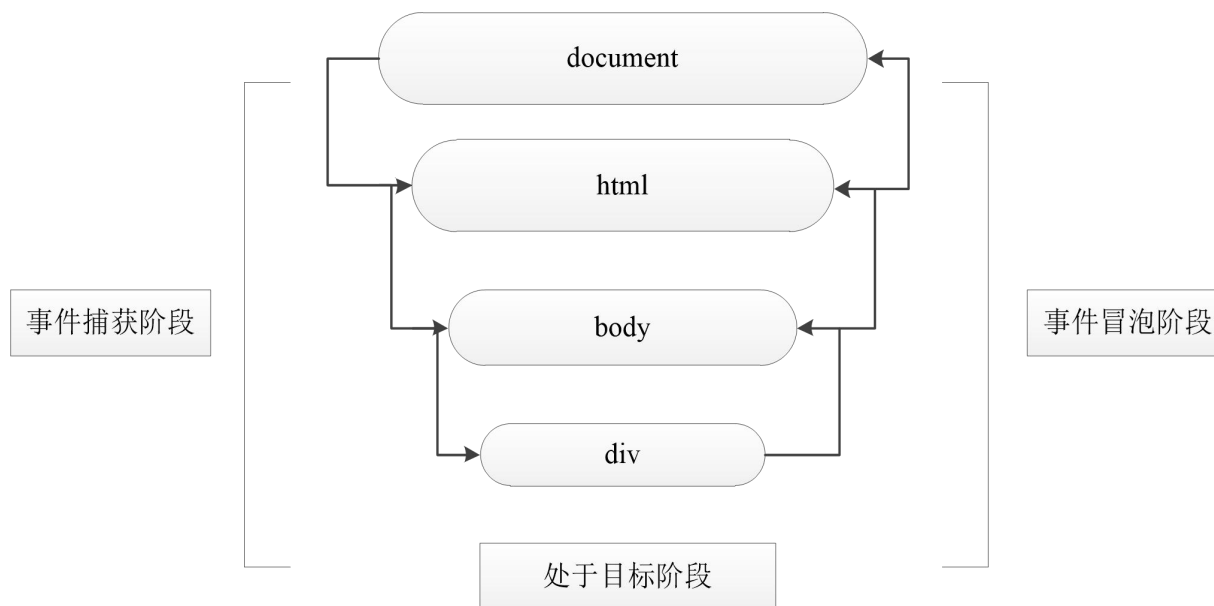
网景（Netscape）公司团队的事件流采用**事件捕获方式**，而**微软**（Microsoft）公司的事件流采用**事件冒泡方式**，**W3C**对网景公司和微软公司提出的方案进行了**中和处理**，规定了事件发生后，**首先实现事件捕获**，但不会对事件进行处理；**然后**进行到**目标阶段**，执行当前元素对象的事件处理程序，但它会被看成是**冒泡阶段**的一部分；最后实现事件的冒泡，逐级对事件进行处理。

7.6 事件进阶

3

DOM事件流

W3C规定的事件流的具体过程对比如下图所示：



W3C规定的事件流方式



7.7 事件对象

1

什么是事件对象

当一个事件发生后，跟事件相关的一系列信息数据的集合都放到这个对象里面，这个对象就是event。只有有了事件event才会存在，它是系统自动创建的，不需要传递参数。



7.7 事件对象

2

事件对象的使用

在标准浏览器中会将一个[event](#)对象直接传入到事件处理程序中，而早期版本的IE浏览器（IE 6~IE 8）中，仅能通过[window.event](#)才能获取事件对象。

语法结构

```
var 事件对象 = window.event           // 早期IE内核浏览器  
DOM对象.事件 = function (event) {} // W3C内核浏览器
```

注意：因为在事件触发时就会产生事件对象，并且系统会以实参的形式传给事件处理函数。所以，在事件处理函数中需要用一個形参来接收事件对象event。



7.7 事件对象

2

事件对象的使用

例7.7.2-2

示例代码

```
<button id="btn">获取event对象</button>
<script>
  var btn = document.getElementById('btn');
  btn.onclick = function(e) {
    var event = e || window.event;  // 获取事件对象的兼容处理
    console.log(event);
  };
</script>
```

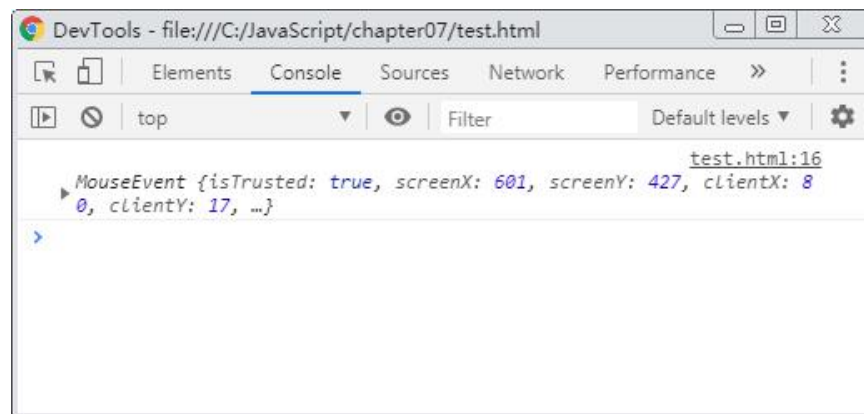
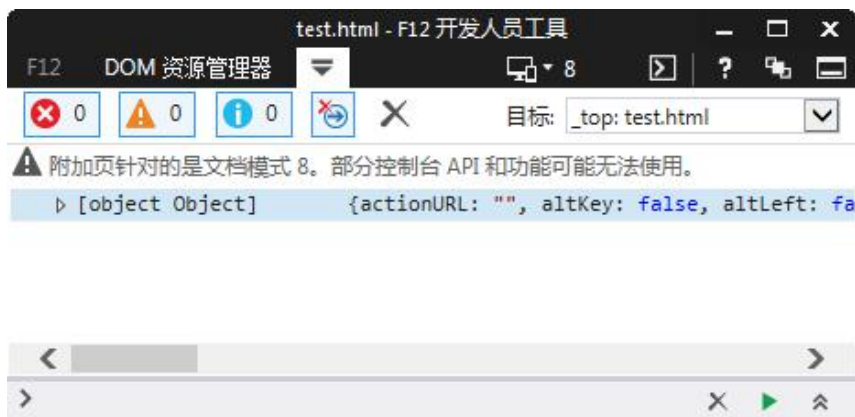


7.7 事件对象

2

事件对象的使用

在IE11的开发人员工具中，通过IE8兼容模式测试，效果如左图所示。在Chrome浏览器中的效果如右图所示：



获取事件对象



7.7 事件对象

3

事件对象的常见属性和方法

在事件发生后，事件对象`event`中会包含一些所有事件都有的属性和方法。所有事件基本上都包括的常用的属性和方法如下表所示。

属性	说明	浏览器
<code>e.target</code>	返回触发事件的对象	标准浏览器
<code>e.srcElement</code>	返回触发事件的对象	非标准IE 6-IE 8使用
<code>e.type</code>	返回事件的类型	所有浏览器
<code>e.stopPropagation()</code>	阻止事件冒泡	标准浏览器
<code>e.cancelBubble</code>	阻止事件冒泡	非标准IE 6-IE 8使用
<code>e.preventDefault()</code>	阻止默认事件（默认行为）	标准浏览器
<code>e.returnValue</code>	阻止默认事件（默认行为）	非标准IE 6-IE 8使用



7.7 事件对象

3

事件对象的常见属性和方法

注意

表中的`type`属性是标准浏览器和早期版本IE浏览器的事件对象的公有属性，通过该属性可以获取发生事件的类型，如`click`、`mouseover`等（不带`on`）。



7.7 事件对象

3

事件对象的常见属性和方法

① 对比e.target和this的区别

在事件处理函数中，e.target返回的是触发事件的对象，而this返回的是绑定事件的对象。因为e.target支持IE9以上，所以编写以下代码，处理兼容性问题。

语法结构

```
div.onclick = function(e) {  
    e = e || window.event;  
    var target = e.target || e.srcElement;  
    console.log(target);  
}
```



7.7 事件对象

3

事件对象的常见属性和方法

② 阻止默认行为

在实际开发中，为了使程序更加严谨，想要确定含有默认行为的标签符合要求后，才能执行默认行为时，可利用事件对象的[`preventDefault\(\)`](#)方法和[`returnValue`](#)属性，禁止所有浏览器执行元素的默认行为。需要注意的是，只有事件对象的[`cancelable`](#)属性设置为[`true`](#)，才可以使用[`preventDefault\(\)`](#)方法取消其默认行为。



7.7 事件对象

3

事件对象的常见属性和方法

例7.7.3-2

案例演示: 下面我们以禁用[a](http://www.baidu.com)标签的链接为例进行演示

语法结构

```
<a href="http://www.baidu.com">百度</a>
<script>
  var a = document.querySelector('a'); // 获取a链接对象
  a.addEventListener('click', function (e) { // 注册单击事件
    e.preventDefault(); // DOM标准写法, 早期版本浏览器不支持
  });
  // 推荐使用传统的注册方式
  a.onclick = function (e) {
    e.preventDefault(); // 标准浏览器使用e.preventDefault()方法
    e.returnValue; // 早期版本浏览器 (IE6、7、8) 使用returnValue属
  }
</script>
```

事件对象的常见属性和方法

可以利用事件对象调用[stopPropagation\(\)](#)方法和设置[cancelBubble](#)属性，实现禁止所有浏览器的事件冒泡行为。

```
if (window.event) { // 早期版本的浏览器
    window.event.cancelBubble = true;
} else { // 标准浏览器
    e.stopPropagation();
}
```



7.6 事件进阶

3

事件对象的常见属性和方法

eg1-冒泡

```
<body>
  <div class="one">
    我是父亲
    <div class="two">
      我是儿子
    </div>
  </div>
  <script type="text/javascript">
    var a = document.querySelector('.one')
      .addEventListener('click', function() {
        console.log('我是父亲')
      }, true)
    var b = document.querySelector('.two')
      .addEventListener('click', function(e) {
        console.log('我是儿子');
        // e.stopPropagation();
      }, true)
  </script>
</body>
```



7.7 事件对象

3

事件对象的常见属性和方法

④ 事件委托

事件委托的原理是，不给每个子节点单独设置事件监听器，而是把事件监听器设置在其父节点上，让其利用事件冒泡的原理影响到每个子节点。

简而言之，就是不给予元素注册事件，给父元素注册事件，让处理代码在父元素的事件中执行。

这样做的的优点在于，只操作了一次DOM，提高了程序的性能。



7.7 事件对象

3

事件对象的常见属性和方法

例7.7.3-4

案例演示：演示事件委托的使用。

语法结构

```
<ul>
  <li>我是第 1 个li</li>
  // ...此处省略多个<li></li>标签
</ul>
<script>
  var ul = document.querySelector('ul');
  ul.addEventListener('click', function (e) {
    e.target.style.backgroundColor = 'pink';
  });
</script>
```



7.8 鼠标事件

1 鼠标事件的常用方法

鼠标是计算机的一种输入设备，也是计算机显示系统纵横坐标定位的指示器，所以[鼠标事件](#)是Web开发中最常用的一类事件。常见的鼠标事件如下表所示：

事件名称	事件触发时机
onclick	单击鼠标左键时触发
onfocus	获得鼠标指针焦点触发
onblur	失去鼠标指针焦点触发
onmouseover	鼠标指针经过时触发
onmouseout	鼠标指针离开时触发
onmousedown	当按下任意鼠标按键时触发
onmouseup	当释放任意鼠标按键时触发
onmousemove	在元素内当鼠标指针移动时持续触发



7.8 鼠标事件

1

鼠标事件的常用方法

例7.8.1-1

① 禁止鼠标右击菜单

contextmenu主要控制应该何时显示上下文菜单，主要应用于程序员取消默认的上下文菜单，示例代码如下。

语法结构

```
document.addEventListener('contextmenu', function (e) {  
    e.preventDefault();  
})
```



7.8 鼠标事件

1

鼠标事件的常用方法

例7.8.1-2

② 禁止鼠标选中

selectstart事件是鼠标开始选择文字时就会触发，如果禁止鼠标选中，需要禁止该事件的默认行为，示例代码如下。

语法结构

```
document.addEventListener('selectstart', function (e) {  
    e.preventDefault();  
})
```



7.8 鼠标事件

2

鼠标事件对象

鼠标事件对象[MouseEvent](#)，是跟鼠标事件相关的一系列信息的集合，可以用来获取当前鼠标的位置信息，鼠标事件位置属性如下表所示。

位置属性（只读）	描述
clientX	鼠标指针位于浏览器页面当前窗口可视区的水平坐标（X轴坐标）
clientY	鼠标指针位于浏览器页面当前窗口可视区的垂直坐标（Y轴坐标）
pageX	鼠标指针位于文档的水平坐标（X轴坐标）， IE 6~IE 8不兼容
pageY	鼠标指针位于文档的垂直坐标（Y轴坐标）， IE 6~IE 8不兼容
screenX	鼠标指针位于屏幕的水平坐标（X轴坐标）
screenY	鼠标指针位于屏幕的垂直坐标（Y轴坐标）



7.8 鼠标事件

2

鼠标事件对象

IE 6~IE 8浏览器中不兼容pageX和pageY属性，在项目开发时需要对IE 6~IE 8浏览器进行兼容处理，示例代码如下所示。

示例代码

```
var pageX = event.pageX || event.clientX +  
(document.body.scrollLeft ||  
document.documentElement.scrollLeft)  
var pageY = event.pageY || event.clientY +  
(document.body.scrollTop ||  
document.documentElement.scrollTop)
```



7.8 鼠标事件

3

【案例】 图片跟随鼠标移动

例7.8.3

案例需求：简单实现图片跟随鼠标移动的效果（不考虑兼容性）。

案例分析：需要使用鼠标指针移动事件`mousemove`，每次鼠标移动，都会获得最新的鼠标指针坐标，把这个`x`和`y`坐标作为图片的`top`和`left` 值就可以实现图片的移动。



7.8 鼠标事件

3

【案例】 图片跟随鼠标移动

① 编写HTML代码

示例代码

```
<style>
img { // 给img图片设置定位
  position: absolute;
  top: 2px;
}
</style>
<body>
   // 需要移动的图片
</body>
```




7.8 鼠标事件

3

【案例】图片跟随鼠标移动

② 编写JS逻辑代码

示例代码

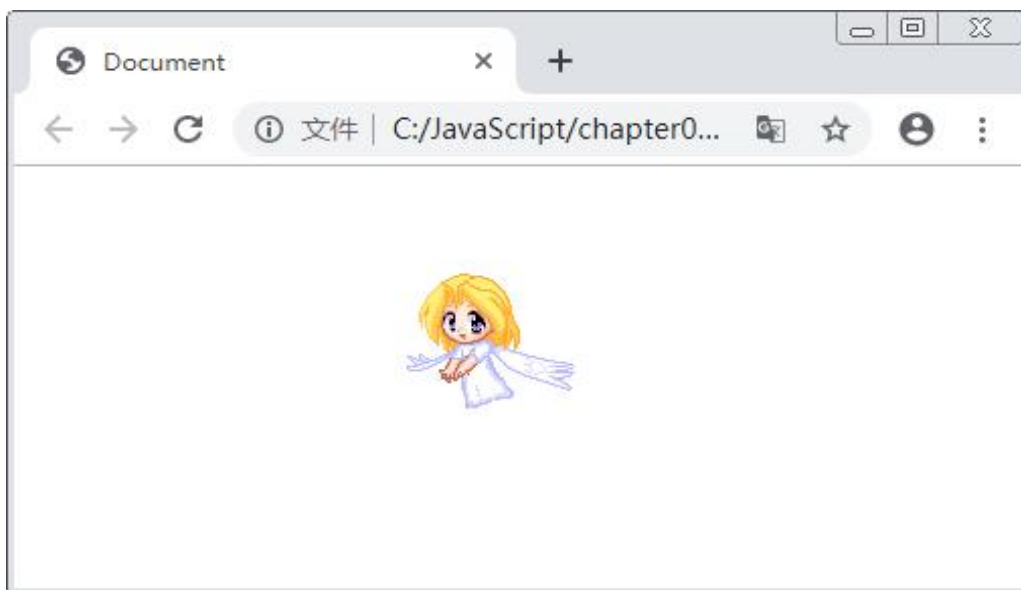
```
<script>
var pic = document.querySelector('img');
document.addEventListener('mousemove', function(e) {
    var x = e.pageX;    // 获取鼠标的X坐标
    var y = e.pageY;    // 获取鼠标的Y坐标
    pic.style.left = x - 50 + 'px';
    pic.style.top = y - 40 + 'px';
});
```

7.8 鼠标事件

3

【案例】 图片跟随鼠标移动

浏览器预览效果如下图所示：



跟随鼠标指针移动的“天使”



7.9 键盘事件

1

键盘事件的常用方法

例7.9.1

键盘事件是指用户在使用键盘时触发的事件。例如，用户按**Esc**键关闭打开的状态栏，按**Enter**键直接完成光标的上下切换等。常用的键盘事件如下表所示。

事件名称	事件触发时机
keypress	某个键盘按键被按下时触发，不识别功能键，如Ctrl、Shift、箭头等
keydown	某个键盘按键被按下时触发
keyup	某个键盘按键被松开时触发



7.9 键盘事件

1

键盘事件的常用方法

注意

keypress事件保存的按键值是ASCII码，keydown和keyup事件保存的按键值是虚拟键码，keydown和keypress如果按住不放的话，会重复触发该对应事件。keyup和keydown事件不区分字母大小写，而keypress区分字母大小写。



7.9 键盘事件

2

键盘事件对象

键盘事件对象[KeyboardEvent](#)，是跟键盘事件相关的一系列信息的集合，根据键盘事件对象中的`keyCode`属性可以得到相应的ASCII码值，进而可以判断用户按下了哪个键。

案例演示：检测用户是否按下了s键，如果按下s 键，就把光标定位到搜索框里面，示例代码如下。

例7.9.2



7.9 键盘事件

2

键盘事件对象

示例代码

```
<body>
  <input type="text">
  <script>
    var search = document.querySelector('input');
    document.addEventListener('keyup', function (e) {
      if (e.keyCode === 83) {
        search.focus();
      }
    });
  </script>
</body>
```



7.9 键盘事件

3

【案例】文本框提示信息

例7.9.3

案例需求：在现实生活中，我们经常会使用快递单号查询功能，查看商品的物流信息状态。有时在用户输入单号时，网站为了让用户看清楚输入的内容，会在文本框上方显示一个提示栏，将用户输入的数字放大。

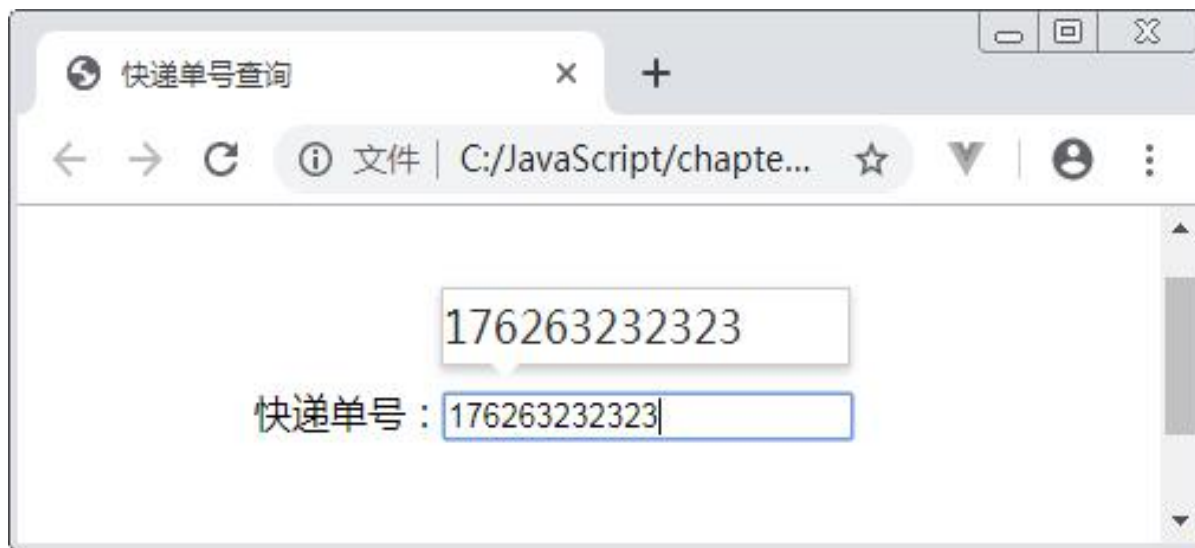
案例分析：当用户在文本框中输入内容时，文本框上面自动显示大号字的内容。如果用户输入为空，需要隐藏大号字内容。

7.9 键盘事件

3

【案例】文本框提示信息

浏览器预览效果如下图所示：



文本框提示效果



7.9 键盘事件

3

【案例】文本框提示信息

① 编写HTML代码

示例代码

```
<body>
  <div class="search">
    <div class="con"> </div>
    <label>快递单号:
      <input type="text" placeholder="请输入您的快递单号" class="num">
    </label>
  </div>
</body>
```



7.9 键盘事件

3

【案例】文本框提示信息

② 检测用户输入，给表单添加键盘事件

示例代码

```
<script>
  var con = document.querySelector('.con');
  var numInput = document.querySelector('.num');
  numInput.addEventListener('keyup', function () {
    if (this.value == '') { con.style.display = 'none';}
    else {
      con.style.display = 'block'; con.innerText = this.value;
    }
  }); // ...在此处编写失去焦点和获得焦点代码
</script>
```



7.9 键盘事件

3

【案例】文本框提示信息

③ 失去焦点，隐藏con元素

示例代码

```
num_input.addEventListener('blur', function() {  
    con.style.display = 'none';  
})
```



7.9 键盘事件

3

【案例】文本框提示信息

④ 获得焦点，显示con元素

示例代码

```
num_input.addEventListener('focus', function() {  
    con.style.display = 'block';  
})
```

本章总结



本章主要讲解了DOM的一些常用操作，以及事件的进阶内容。通过本章的学习，读者应掌握如何进行排他操作、属性操作、节点操作，学会如何创建节点、添加节点、删除节点、复制节点。在事件进阶部分，要掌握事件对象、鼠标事件对象、键盘事件对象及各事件常用方法和属性，能够通过鼠标及键盘操作元素。